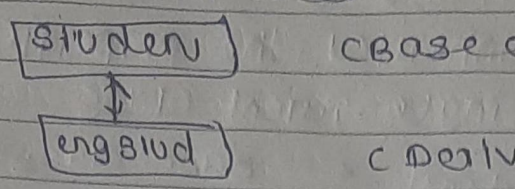
Page No.
Date

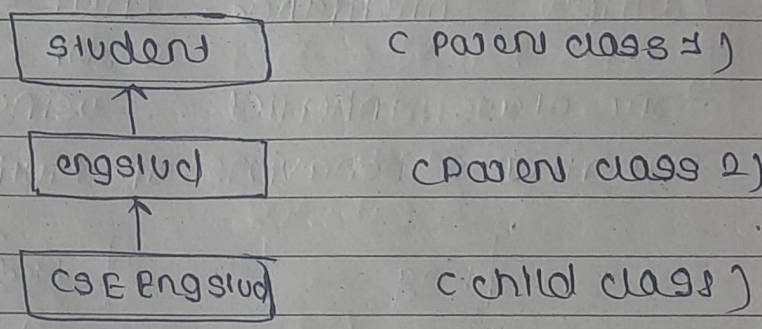
1. Single Inheritance



- one class inherits properties and behaviours of one parent class (Base class)

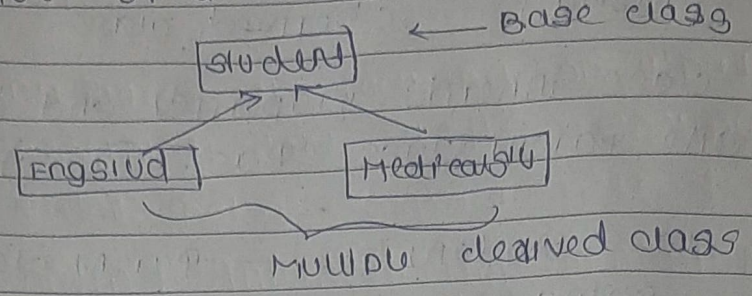
eg: class student { — parent class
    string name;
    int age;
    void markAttend () {
    }
    class Engstud extends student { — childclass
    }
}

2. Multi-Level Inheritance



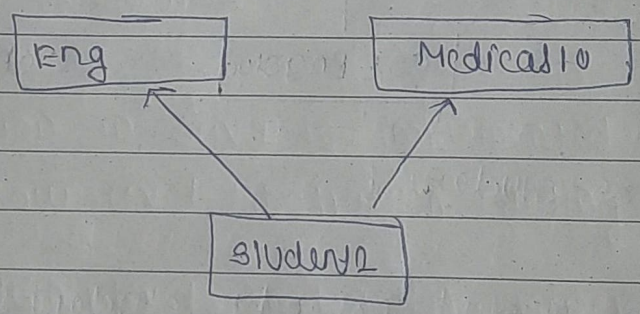
eg: class student {
    }
  class engstud extends student {
    }
  class CSE Eng extends engstud {
    }
}

3. Hierarchical Inheritance
Multiple classes inherits the properties and behaviour of a single parent class

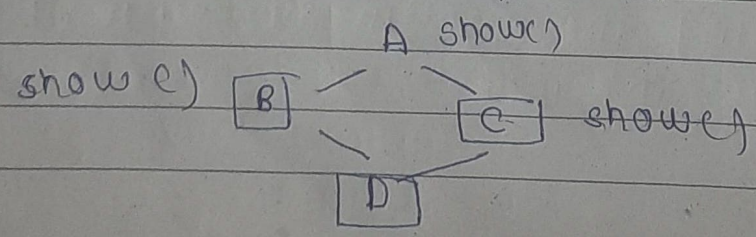



```
class student
{
}
class Engstud Extends student {
}
class Medicals Extends student {
}
```

4. Multiple Inheritance (not supported in Java)
↳ one class inherits properties & behaviors of multiple classes but can be acheived using Interface

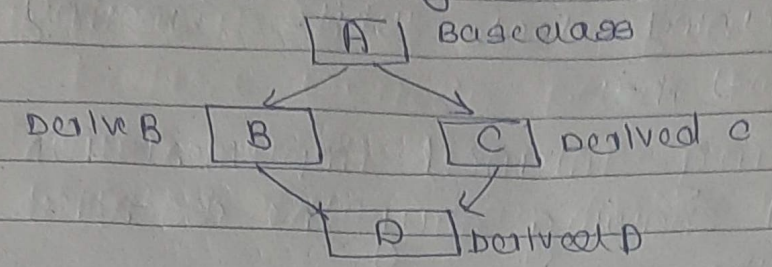


Why Multiple Inheritance not supported in Java?
→ Due to diamond problem



5. Hybrid Inheritance

- A combination of two or more types of inheritances.
- Not supported in Java directly, but can be achieved using interfaces.



* Super keyword

super → holds Reference of parent obj
↓
this → Reference of current obj

- super keyword is a reference variable in Java.
- It is used to refer to the immediate parent class object (i.e. superclass of current obj)

* Uses of super keyword
  - ↳ To access parent class variable.
  - ↳ To call parent class Method
  - ↳ To call parent class constructor.

Page No.
Date

Abstraction & Polymorphism | Abstract class vs Inter

Abstraction

↳ The process of focusing on what something does, while ignoring how it does that.

↳ hiding internal implementation details and showing only the essential features to the user.

Real World Example:

① when you drive a car, you only need to know how to operate the steering wheel, pedals and gear shift. You don't need to understand how engine works.

② ATM Machine does what
↳ deposite
↳ withdraw
↳ check Balance

* Abstraction in Java is implemented in two ways
↳ ① Low-level Abstraction (hiding implem details)
↳ ② High-level Abstraction (seperate what from How)

Low-level Abstraction

```
class car {
  String type;

  start() {
  }
  accelerate() {
  }
  brake() {
  }
}
```

```
main() {
  car c = new car();
  c.start();
  c.accelerate();
  c.brake();
}
```

- How to acheive Abstraction -
  we can acheive abstraction by two ways
  - using Abstract class
  - using Interface

* Abstract class

Abstract Methods
- It is a method that is declared without an implementation (no method body)
- It only provides Method declaration. & forces subclasses to Provide actual Implementation.
- Declared using abstract keyword

syntax : abstract returnType MethodName (parameters);
eg : abstract void accelerate();

Rules of abstract Methods
- No Method body - ends with semicolon (;)
- must be declared inside an abstract class or interface.

- Abstract methods must be overridden by subclasses, unless subclass is also abstract
- cannot be private, static or final.

Abstract class

- An abstract class is a class that is declared using abstract method.
  - It can contain a mix abstract methods (without body) and concrete methods (with body)
  - An Abstract class cannot be instantiated (you cannot create objects of it).

eg:

```
abstract class car {
    start() {-}
    abstract accelerate();
    abstract brake();
}
```

concrete class →

```
class EC extend car
{ accelerate()
}
```

subclass

```
class Nexon extend car
{ accelerate()
}
```

subclass

```
car c = new EC car
car c = new Nexon
```

* Concrete class:
  - A class that is not abstract
  - has complete implementation
  - can be instantiated. eg: class car { }
* Concrete Method
  - has a body
  - defines what method actually does

eg: void start() {
    s.o.p(n c "car is starting");
}